

AI Business Knowledge Assistant — System Overview

A working prototype of a business knowledge assistant that answers free-form questions by retrieving and combining information across multiple data sources — **PDF documents** and a **structured database** — and returns a **grounded answer with citations you can trace back to the exact source**. It works in **English and Hebrew**.

This document walks through what the system does, how it is built, and what each part looks like in practice, with screenshots from the running application.

What it does

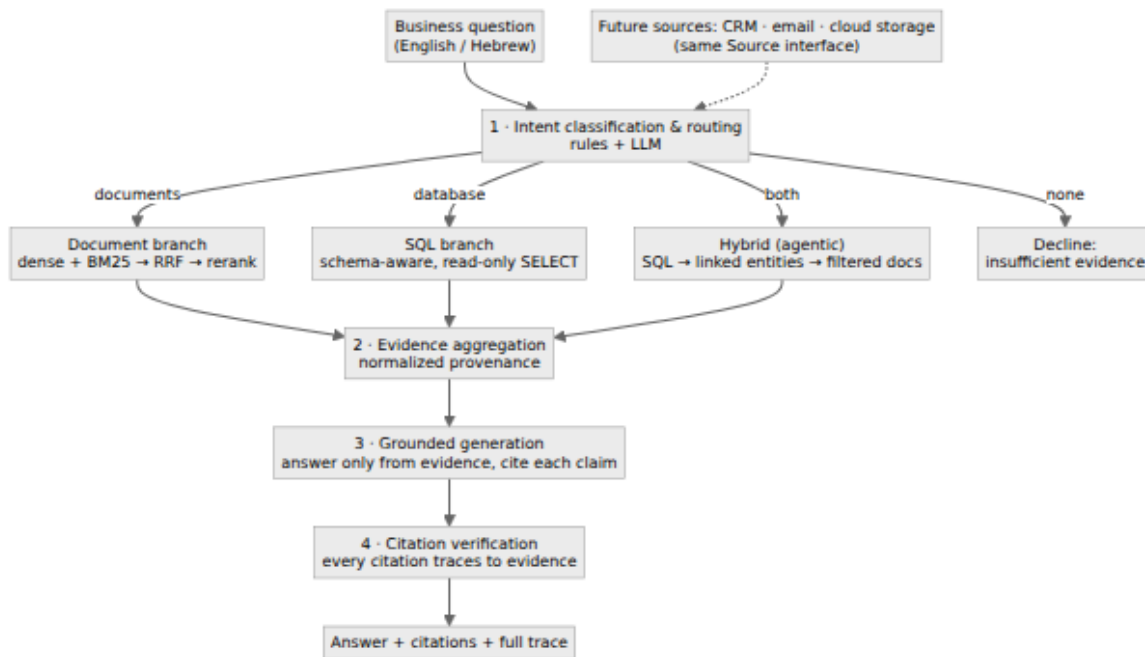
Most "chat with your documents" tools do one thing: embed PDFs and run a similarity search. That breaks down for real business questions, which usually need **structured data and documents together** — for example, *"which customers are overdue, and what do their contracts allow us to do about it?"*

This assistant treats the problem as **retrieval orchestration**. For every question it:

1. **Decides which source(s) are relevant** — the documents, the database, or both.
2. **Retrieves from each appropriately** — semantic + keyword search over documents; schema-aware, read-only SQL over the database.
3. **Combines the evidence** and writes an answer that uses **only** that evidence.
4. **Cites every claim** and verifies each citation against what was actually retrieved.

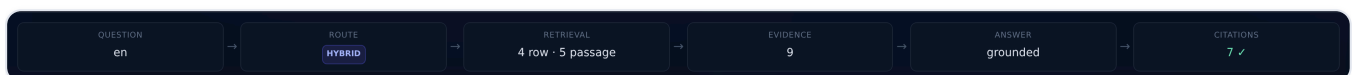
If no source can answer, it says so rather than guessing.

How it works



Every question flows through four stages — routing, retrieval, grounded generation, and citation verification — and the application exposes all of them for inspection. The design is source-agnostic: new sources (CRM, email, cloud storage) implement the same interface and become available to the router without changing the pipeline.

The application surfaces this as a clear pipeline on every answer:



Walkthrough

1. A question that spans documents *and* the database

"Which customers have overdue invoices, and what do their agreements say about service suspension?"

This cannot be answered by document search alone. The system queries the database for overdue customers, links each customer to *their specific contract*, retrieves the relevant clause from only those documents, and writes a combined answer — each line cited to either a database record or a contract page:

ANSWER

mode: live

Customers with overdue invoices are Acme Corporation, Globex Industries, and Initech LLC.
- Acme Corporation: Services may be suspended if an invoice is unpaid for more than 30 days after its due date, with a 5-day notice before suspension e5.
- Globex Industries: Services may be suspended if an invoice is unpaid for more than 45 days after its due date, with a 5-day notice before suspension e7.
- Initech LLC: Services may be suspended if an invoice is unpaid for more than 30 days after its due date, with a 5-day notice before suspension e6.

CITATIONS

e1 [business_db: invoices INV-1187] e2 [business_db: invoices INV-1201] e3 [business_db: invoices INV-1190] e4 [business_db: invoices INV-1175] e5 [ACME_MSA_2025.pdf p.1] e6 [INITECH_Agreement.pdf p.11] e7 [GLOBEX_Service_Agreement.pdf p.1]

The key point is that **you can see exactly how it reached that answer**. The orchestration trace shows the actual steps it took:

Orchestrator trace

4 steps

1. Router → HYBRID (agentic: SQL → entities → filtered documents). The question requires identifying customers with overdue invoices from the database and then looking up the relevant clauses regarding service suspension in their agreements from the documents.

2. Step 1 → SQL returned 4 row(s)

3. Step 2 → linked 3 customer(s) ['Acme Corporation', 'Globex Industries', 'Initech LLC'] → 3 contract document(s); restricting document retrieval to them.

4. Step 3 → document retrieval (filtered to 3 doc(s)) selected 5 passage(s).

The database step shows the exact query that ran (validated as read-only before execution) and the rows it returned:

SQL branch

2 query

sql_step

validated · read-only

4 rows

0.5 ms

tables: customers, invoices

SELECT invoices.customer_id, customers.name, invoices.invoice_ref, invoices.due_date, invoices.status FROM invoices JOIN customers ON invoices.customer_id = customers.id WHERE invoices.due_date < DATE() AND invoices.status = 'overdue' LIMIT 200

customer_id	name	invoice_ref	due_date	status
1	Acme Corporation	INV-1187	2026-05-20	overdue
1	Acme Corporation	INV-1201	2026-05-31	overdue
2	Globex Industries	INV-1190	2026-05-25	overdue
3	Initech LLC	INV-1175	2026-05-10	overdue

entity_link

validated · read-only

3 rows

0.3 ms

tables: contracts, customers

SELECT c.name AS customer, ct.pdf_file FROM contracts AS ct JOIN customers AS c ON c.id = ct.customer_id WHERE ct.customer_id IN (1, 2, 3) LIMIT 200

customer	pdf_file
Acme Corporation	ACME_MSA_2025.pdf
Globex Industries	GLOBEX_Service_Agreement.pdf
Initech LLC	INITECH_Agreement.pdf

And the document step shows every candidate passage with its retrieval scores, and which ones were selected — restricted to just the relevant customers' contracts:

Document retrieval — dense + BM25 → RRF → rerank

20 candidates

embed: openai/text-embedding-3-small

rerank: none (fusion order)

dense_top_k: 20

bm25_top_k: 20

rrf_k: 60

rerank_top_n: 20

final_k: 5

vector_backend: numpy

filtered → 3 doc(s)

#	document	p.	dense	bm25	rrf	rerank	
1	ACME_MSA_2025.pdf · 5. Service Suspension	1	#1	#4	0.932	—	selected
2	INITECH_Agreement.pdf · 5. Service Suspension	1	#2	#6	0.931	—	selected
3	GLOBEX_Service_Agreement.pdf · 5. Service Suspension	1	#3	#5	0.931	—	selected
4	INITECH_Agreement.pdf · 6. Termination and Penalties	1	#4	#10	0.930	—	selected
5	GLOBEX_Service_Agreement.pdf · 6. Termination and Penalties	1	#5	#9	0.930	—	selected
6	ACME_MSA_2025.pdf · 6. Termination and Penalties	1	#6	#8	0.930	—	
7	GLOBEX_Service_Agreement.pdf · Preamble	1	#7	#7	0.930	—	
8	ACME_MSA_2025.pdf · 4. Service Levels (SLA-2025)	1	#16	#1	0.930	—	
9	GLOBEX_Service_Agreement.pdf · 4. Service Levels (SLA-2025)	1	#15	#2	0.929	—	
10	INITECH_Agreement.pdf · 4. Service Levels (SLA-2025)	1	#14	#3	0.929	—	

2. Asking the database directly

"What is the total outstanding invoice amount per customer?"

The system recognises this as a pure data question, generates a read-only SQL query, and returns the result with each row cited:

ANSWER mode: live
Acme Corporation: 60,000.0, Globex Industries: 30,000.0, Initech LLC: 15,000.0

CITATIONS
e1 [business_db: invoices Acme Corporation] e2 [business_db: invoices Globex Industries] e3 [business_db: invoices Initech LLC]

SQL branch 1 query

sql_main validated read-only 3 rows 0.5 ms tables: customers, invoices

```
SELECT invoices.customer_id, SUM(invoices.amount_usd) AS total_outstanding, customers.name FROM invoices JOIN customers ON invoices.customer_id = customers.id WHERE invoices.status = 'overdue' GROUP BY invoices.customer_id LIMIT 200
```

customer_id	total_outstanding	name
1	60000	Acme Corporation
2	30000	Globex Industries
3	15000	Initech LLC

3. Working in Hebrew

"מה אומר ההסכם של תבור מערכות על השעיית שירות וקנסות?" ("What does the Tavor Systems agreement say about service suspension and penalties?")

The same pipeline handles Hebrew end to end — it routes the question to the Hebrew contract, answers in Hebrew with right-to-left formatting, and cites the source:

ANSWER mode: live

ההסכם של תבור מערכות קובע כי נותן השירות רשאי להשעות את השירותים אם חשבונית שאינה שנויה במחלוקת לא שולמה במשך למעלה מ-45 יום ממועד הפירעון, ובכפוף להודעה מוקדמת של 5 ימי עסקים. השירות יחודש בתוך 2 ימי עסקים ממועד התשלום המלא. בנוסף, במקרה של סיום ההסכם על ידי הלקוח מטעמי נוחות לפני תום התקופה, ישלם הלקוח קנס יציאה בשיעור 12% מיתרת שווי ההסכם, בנוסף לכל סכום שטרם שולם.

CITATIONS
e3 [TAVOR_Contract_HE.pdf p.1] e5 [TAVOR_Contract_HE.pdf p.1]

4. Knowing when *not* to answer

"What is our employee headcount in Berlin?"

Nothing in the documents or the database can answer this. Instead of inventing a number, the system declines:

ANSWER

mode: live

⚠ insufficient evidence — not answered

Insufficient evidence: no relevant records or document passages were retrieved from the available sources to answer this question.

Why every answer is auditable

The thing that makes this trustworthy for business use is **traceability**. For any answer, you can open the panel beneath it and see:

- **The routing decision** — why the question went where it did, and how confident the system was.
- **The exact SQL** — generated, validated as read-only, and the rows it returned.
- **The retrieval detail** — every candidate document passage with its keyword and semantic ranking, and which were selected.
- **The evidence** — each item showing its source document and page, or its database table and row.
- **A citation check** — confirming every citation in the answer traces back to retrieved evidence.
- **Timing and cost** — how long each stage took and what the answer cost.

Clicking any citation in an answer highlights the exact passage or record it came from.

Extending to other sources

The system is built around a single source interface. The two sources in this prototype — the contract documents and the business database — implement it, and so would future **CRM, email, or cloud-storage** integrations. A new source only needs to describe what it can answer and how to retrieve from it; the routing and answer-

generation logic stays the same. (The repository includes a placeholder CRM connector to illustrate the pattern.)

This means the assistant can grow from two sources to many without re-architecting.

Running it

The whole system runs with a single command:

```
docker compose up --build
```

- The application UI is at `http://localhost:3000`.
- It supports any modern LLM provider (it was demonstrated here on OpenAI; it also runs on Anthropic, or fully local models), and it can run in a no-key demonstration mode.

The source code, architecture notes, and sample data are in the repository.

Prototype prepared by Apoorv. Sample data is synthetic. Happy to walk through any part of the architecture or the retrieval logic in more detail.